

List of Experiments

S.No	Name of the Experiment	COs
1.	Implement singly and doubly linked lists with insertion, deletion, and search.	CO1
2.	Implement a stack using array or linked list. Use it to evaluate postfix expressions.	CO1
3.	Implement linear and circular queues; simulate a printer or task scheduler.	CO1
4.	Implement factorial, Fibonacci, and binary search using both recursion and iteration.	CO1
5.	Develop a program that checks if parentheses in an expression are balanced	CO1
6.	Implement Binary Search Tree (BST) and perform inorder, preorder, postorder traversal.	CO2
7.	Build an expression tree from a postfix expression and evaluate it.	CO2
8.	Implement a priority queue using min-heap or max-heap.	CO2
9.	Implement Dijkstra's algorithm for shortest path detection.	CO3
10.	Implement Prim's and Kruskal's algorithms to find MST.	CO3
11.	Implement and compare quick sort, merge sort, and heap sort.	CO4
12.	Implement binary search and compare with linear search.	CO4
13.	Implement a hash table with chaining or open addressing	CO5
14.	To solve string analysis problems using hash structures.	CO5
15.	Implement first-fit and best-fit memory allocation strategies	CO5

Ex.No: 1 Implement singly and doubly linked lists with insertion, deletion, and search.

Aim: To implement **Singly Linked List** and **Doubly Linked List** in Python and perform the following operations Insertion, Deletion, Search and Display the list

Algorithm / Procedure

Singly Linked List

1. Define a Node class with data and next pointer.
2. Create a Singly Linked List class.
3. Implement the following methods:
 - o Insert a node at the end. o Delete a node by value. o Search for a node.
 - o Display all nodes.
4. Create the linked list object.
5. Insert elements into the list.
6. Display the list.
7. Search for an element.
8. Delete an element.
9. Display the updated list.

Doubly Linked List

1. Define a Node class with data, prev, and next pointers.
2. Create a DoublyLinkedList class.
3. Implement the following methods:
 - o Insert at the end. o Delete a node. o Search a node.
 - o Display the list.
4. Create the linked list.
5. Insert elements.
6. Display the list.

7. Search for an element.
8. Delete an element.
9. Display the updated list.

Program

```
# -----
# Singly Linked List # ---
-----

class SNode:
    def __init__(self, data):
self.data = data      self.next = None
class SinglyLinkedList:  def
__init__(self):      self.head = None
def insert(self, data):      new_node
= SNode(data)      if self.head is
None:      self.head = new_node
return      temp = self.head
while temp.next:      temp =
temp.next      temp.next = new_node
def delete(self, key):      temp =
self.head      if temp and temp.data
== key:      self.head = temp.next
return      prev = None      while
temp and temp.data != key:
        prev = temp
temp = temp.next      if
temp is None:
print(key, "not found")
return      prev.next =
temp.next  def search(self,
key):      temp = self.head
while temp:      if
temp.data == key:
return True      temp =
temp.next      return False
def display(self):      temp =
self.head      while temp:
print(temp.data, end=" -> ")
temp = temp.next
print("None") # -----
----- # Doubly Linked
List
# -----
class DNode:  def
__init__(self, data):
```

```

self.data = data      self.prev
= None      self.next = None
class DoublyLinkedList:  def
__init__(self):      self.head =
None  def insert(self, data):
new_node = DNode(data)
if self.head is None:
self.head = new_node
return      temp = self.head
while temp.next:      temp
= temp.next      temp.next =
new_node
new_node.prev = temp  def
delete(self, key):      temp =
self.head      while temp:
        if temp.data == key:
if temp.prev:
        temp.prev.next = temp.next
else:
        self.head = temp.next
if temp.next:
temp.next.prev = temp.prev
return      temp = temp.next
print(key, "not found")  def
search(self, key):      temp = self.head
while temp:      if temp.data ==
key:      return True      temp
= temp.next      return False  def
display(self):      temp = self.head
        while temp:
            print(temp.data, end=" <-> ")
temp = temp.next      print("None")
# -----
# Main Program
# -----
print("SINGLY LINKED LIST")
sll = SinglyLinkedList()
sll.insert(10) sll.insert(20)
sll.insert(30) sll.insert(40)
print("Linked List:") sll.display()
print("Search 30:", sll.search(30))
sll.delete(20) print("After
deleting 20:") sll.display()
print("\nDOUBLY LINKED LIST")
dll = DoublyLinkedList()
dll.insert(100) dll.insert(200)

```

```
dll.insert(300) dll.insert(400)
print("Linked List:") dll.display()
print("Search 300:", dll.search(300))
dll.delete(200) print("After deleting
200:") dll.display()
```

Result

The Singly Linked List and Doubly Linked List were successfully implemented in Python. The operations insertion, deletion, search, and display were performed successfully, and the linked lists were updated correctly after each operation.

Ex.No : 2 Implement a stack using array or linked list. Use it to evaluate postfix expressions

Aim

To implement a **Stack** using an array (Python list) and use it to **evaluate a postfix expression**.

Procedure

1. Create an empty list to represent the stack.
2. Define the following stack operations:
 - o **Push:** Insert an element onto the top of the stack.
 - o **Pop:** Remove and return the top element from the stack.
3. Read the postfix expression from the user.
4. Scan the postfix expression from left to right.
5. For each symbol in the expression:
 - o If the symbol is an **operand (digit)**, push it onto the stack.
 - o If the symbol is an **operator (+, -, *, /, %, ^)**:

- Pop the top two operands from the stack.
 - Perform the required operation.
 - Push the result back onto the stack.
6. Continue until all symbols are processed.
 7. The final value remaining in the stack is the result of the postfix expression.
 8. Display the evaluated result.

Program:

Stack Implementation Using Array

```
class Stack:
    def __init__(self):
        self.stack = []
    def push(self, item):
        self.stack.append(item)
    def pop(self):
        if len(self.stack) == 0:
            return None
        return self.stack.pop()

# Function to evaluate postfix expression
def evaluate_postfix(expression):
    s = Stack()
    for ch in expression:
        if ch.isdigit():
            s.push(int(ch))
        else:
            b = s.pop()
            a = s.pop()
            if ch == '+':
                s.push(a + b)
            elif ch == '-':
                s.push(a - b)
            elif ch == '*':
```

```

        s.push(a * b)
elif ch == '/':
        s.push(a / b)
elif ch == '%':
        s.push(a % b)
elif ch == '^':
        s.push(a ** b)    return s.pop()
# Main Program
expr = input("Enter postfix expression: ")
result = evaluate_postfix(expr)
print("Result =", result)

```

Result

The Stack was successfully implemented using an array (Python list). The stack operations (push and pop) were used to evaluate the given postfix expression, and the correct result was obtained successfully.

Ex.No : 3 Implement linear and circular queues; simulate a printer or task scheduler.

Aim

To implement **Linear Queue** and **Circular Queue** using Python and simulate a **printer queue** or **task scheduler** using queue operations.

Procedure

Linear Queue

1. Create an empty queue using a Python list.
2. Implement the following operations:
 - o **Enqueue:** Add a task to the rear of the queue. o **Dequeue:** Remove a task from the front of the queue.
 - o **Display:** Show all tasks in the queue.
3. Insert print jobs into the queue.
4. Process the jobs in the order they were added (FIFO).
5. Display the remaining jobs after each operation.

Circular Queue

1. Create a fixed-size array.
2. Initialize front and rear to -1.
3. Perform the following operations:
 - o **Enqueue:** Insert a task if the queue is not full.
 - o **Dequeue:** Remove a task if the queue is not empty.
 - o **Display:** Show the elements from front to rear.
4. Simulate a task scheduler where completed tasks are removed and new tasks are added, utilizing the circular nature of the queue.

Program:

```
# -----  
# Linear Queue (Printer Simulation)  
# ----- class  
  
LinearQueue:  
  
    def __init__(self):        self.queue = []  
    def enqueue(self, item):  
self.queue.append(item)        print(item,  
"added to printer queue.")    def  
dequeue(self):                if len(self.queue) ==  
0:  
        print("Printer Queue is Empty")  
    else:  
        print(self.queue.pop(0), "printed successfully.")  
    def display(self):        if len(self.queue) == 0:  
        print("Printer Queue is Empty")  
    else:  
        print("Printer Queue:", self.queue)  
print("LINEAR QUEUE - Printer Simulation")  
pq = LinearQueue()  
pq.enqueue("Document1")
```



```

pq.enqueue("Document2")
pq.enqueue("Document3") pq.display()
pq.dequeue() pq.display()
# ----- #

Circular Queue (Task Scheduler)
# -----

class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, item):
        if (self.rear + 1) % self.size == self.front:
            print("Task Queue is Full")
            return
        if self.front == -1:
            self.front = 0
            self.rear = 0
        else:
            self.rear = (self.rear + 1) % self.size
        self.queue[self.rear] = item
        print(item, "scheduled.")

    def dequeue(self):
        if self.front == -1:
            print("Task Queue is Empty")
            return
        print(self.queue[self.front], "completed.")
        if self.front == self.rear:
            self.front = -1
        else:
            self.front = (self.front + 1) % self.size

    def display(self):
        if self.front == -1:
            print("Task Queue is Empty")

```

```

        return

    print("Scheduled Tasks:", end=" ")

    i = self.front    while True:

        print(self.queue[i], end=" ")

    if i == self.rear:
        break        i = (i
+ 1) % self.size    print()

print("\nCIRCULAR QUEUE - Task Scheduler")

cq = CircularQueue(5) cq.enqueue("Task1")
cq.enqueue("Task2") cq.enqueue("Task3")
cq.display() cq.dequeue() cq.enqueue("Task4")
cq.enqueue("Task5") cq.display()

```

Result

The **Linear Queue** and **Circular Queue** were successfully implemented in Python. Queue operations such as **enqueue**, **dequeue**, and **display** were performed correctly. The **Linear Queue** successfully simulated a **printer queue**, while the **Circular Queue** effectively simulated a **task scheduler** following the **First-In-First-Out (FIFO)** principle.

Ex.No: 4 Implement factorial, Fibonacci, and binary search using both recursion and iteration.

Aim

To implement **Factorial**, **Fibonacci**, and **Binary Search** using both **iterative** and **recursive** approaches in Python and compare their outputs.

Procedure

Factorial

1. Read a positive integer from the user.
2. Compute the factorial using an **iterative method** with a loop.
3. Compute the factorial using a **recursive function**.
4. Display both results.

Fibonacci

1. Read the number of terms from the user.
2. Generate the Fibonacci sequence using an **iterative method**.
3. Generate the Fibonacci sequence using a **recursive function**.
4. Display both sequences.

Binary Search

1. Create a sorted list of elements.
2. Read the element to be searched.
3. Search the element using **iterative binary search**.
4. Search the element using **recursive binary search**.
5. Display the position of the element if found; otherwise, display a suitable message.

Program:

```
# -----  
  
# Factorial  
  
# ----- def  
factorial_iterative(n):  
    fact = 1  
    for i in range(1, n + 1):
```

```

        fact *= i    return
fact def
factorial_recursive(n):
if n == 0 or n == 1:
    return 1    return n *
factorial_recursive(n - 1)
# -----
# Fibonacci
# ----- def
fibonacci_iterative(n):
    a, b = 0, 1    series = []    for i in range(n):
series.append(a)    a, b = b, a + b    return series def
fibonacci_recursive(n):    if n <= 1:    return n    return
fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2) def
fibonacci_series_recursive(n):
    series = []    for i in range(n):
series.append(fibonacci_recursive(i))
return series
# -----
# Binary Search
# ----- def
binary_search_iterative(arr, key):
low = 0    high = len(arr) - 1
while low <= high:
    mid = (low + high) // 2
if arr[mid] == key:
    return mid    elif arr[mid] < key:
low = mid + 1    else:    high = mid - 1
return -1 def binary_search_recursive(arr, low,

```

```

high, key):    if low > high:        return -1
mid = (low + high) // 2    if arr[mid] == key:
    return mid
elif arr[mid] < key:
    return binary_search_recursive(arr, mid + 1, high, key)
else:
    return binary_search_recursive(arr, low, mid - 1, key)
# -----
# Main Program
# ----- n = int(input("Enter a number for
factorial: ")) print("Iterative Factorial =", factorial_iterative(n))
print("Recursive Factorial =", factorial_recursive(n)) terms =
int(input("\nEnter number of Fibonacci terms: "))
print("Iterative Fibonacci:", fibonacci_iterative(terms))
print("Recursive Fibonacci:", fibonacci_series_recursive(terms))
arr = [10, 20, 30, 40, 50, 60, 70, 80] key = int(input("\nEnter
element to search: ")) pos1 = binary_search_iterative(arr, key)
pos2 = binary_search_recursive(arr, 0, len(arr) - 1, key) if pos1
!= -1:
    print("Iterative Binary Search: Element found at index", pos1) else:
    print("Iterative Binary Search: Element not found") if
pos2 != -1:
    print("Recursive Binary Search: Element found at index", pos2) else:
    print("Recursive Binary Search: Element not found")

```

Result

The **Factorial**, **Fibonacci**, and **Binary Search** algorithms were successfully implemented using both **iterative** and **recursive** methods in Python. The outputs obtained from both approaches were identical, demonstrating the correctness of the implementations.

Ex.No: 5 Develop a program that checks if parentheses in an expression are balanced

Aim

To develop a Python program that checks whether the **parentheses, braces, and brackets** in a given expression are **balanced** using a **stack**.

Procedure

1. Create an empty stack using a Python list.
2. Read an expression from the user.
3. Traverse each character of the expression.
4. If the character is an opening bracket ((, {, [), push it onto the stack.
5. If the character is a closing bracket (), },]):
 - o Check whether the stack is empty. If it is, the expression is not balanced.
 - o Otherwise, pop the top element from the stack.
 - o Verify that the popped opening bracket matches the current closing bracket.

6. After scanning the entire expression:
 - o If the stack is empty, the expression is balanced.
 - o Otherwise, the expression is not balanced.
7. Display the result.

Program:

```
# Function to check balanced parentheses
def is_balanced(expression):
    stack = []
    pairs = {
        ')': '(',
        '}': '{',
        ']': '['
    }
    for ch in expression:
        if ch in "({[":
            stack.append(ch)
        elif ch in ")}]":
            if len(stack) == 0:
                return False
            top = stack.pop()
            if top != pairs[ch]:
                return False
    return len(stack) == 0 #

Main Program
expr = input("Enter an expression: ")
if is_balanced(expr):
    print("The expression is Balanced.")
else:
    print("The expression is Not Balanced.")
```

Result

The Python program was successfully developed to check whether the **parentheses, brackets, and braces** in a given expression are balanced using a **stack**. The program correctly identifies both **balanced** and **unbalanced** expressions.

Ex.No: 6 Implement Binary Search Tree (BST) and perform inorder, preorder, postorder traversal

Aim

To implement a Binary Search Tree (BST) in Python and perform the following tree traversals Inorder Traversal, Preorder Traversal and Postorder Traversal **Procedure**

1. Define a Node class with data, left child, and right child.
2. Create a Binary Search Tree (BST).
3. Insert nodes into the BST according to BST properties:
 - o If the new value is less than the current node, insert it into the left subtree.
 - o If the new value is greater than the current node, insert it into the right subtree.
4. Implement **Inorder Traversal** (Left → Root → Right).
5. Implement **Preorder Traversal** (Root → Left → Right).
6. Implement **Postorder Traversal** (Left → Right → Root).
7. Insert sample values into the BST.
8. Display the traversal results.

Program

```
# Binary Search Tree Implementation

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None

    # Insert a node
    def insert(self, root, data):
        if root is None:
            return Node(data)
        if data < root.data:
            root.left = self.insert(root.left, data)
        else:
            root.right = self.insert(root.right, data)
        return root

    # Inorder Traversal
    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)

    # Preorder Traversal
    def preorder(self, root):
        if root:
            print(root.data, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)
        #
```

```

Postorder Traversal    def
postorder(self, root):
    if root:
        self.postorder(root.left)
    self.postorder(root.right)    print(root.data,
    end=" ")
# Main Program tree = BST()
values = [50, 30, 70, 20, 40, 60, 80]
for value in values:
    tree.root = tree.insert(tree.root, value) print("Inorder Traversal:") tree.inorder(tree.root)
print("\n\nPreorder Traversal:") tree.preorder(tree.root) print("\n\nPostorder Traversal:")
tree.postorder(tree.root)

```

Result

The **Binary Search Tree (BST)** was successfully implemented in Python. The tree was constructed by inserting nodes according to BST properties, and the **Inorder**, **Preorder**, and **Postorder** traversals were performed successfully, producing the expected traversal sequences.

Ex.No: 7 Build an expression tree from a postfix expression and evaluate it.

Aim

To construct an **Expression Tree** from a given **postfix expression** and evaluate the expression using tree traversal in Python.

Procedure

1. Define a node structure containing:
 - o Data o Left child o Right child
2. Create an empty stack.
3. Scan the postfix expression from left to right.
4. For each symbol:
 - o If it is an operand, create a node and push it onto the stack.
 - o If it is an operator:
 - Pop two nodes from the stack.
 - Create a new node with the operator.
 - Make the first popped node the right child and the second popped node the left child.
 - Push the new node back onto the stack.

5. After processing all symbols, the remaining node in the stack is the root of the expression tree.
6. Recursively evaluate the expression tree:
 - o Return the operand if the node is a leaf.
 - o Otherwise, evaluate the left and right subtrees and apply the operator.
7. Display the result.

Program:

Expression Tree Node

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

# Build Expression Tree
def build_tree(postfix):
    stack = []
    operators = "+-*/"
    for ch in postfix:
        if ch.isdigit():
            stack.append(Node(ch))
        elif ch in operators:
            node = Node(ch)
            right = stack.pop()
            left = stack.pop()
            node.left = left
            node.right = right
            stack.append(node)
    return stack.pop()

# Evaluate Expression Tree
def evaluate(root):
    if root.left is None and root.right is None:
        return int(root.value)
    left = evaluate(root.left)
    right = evaluate(root.right)
```

```

if root.value == '+':
    return left + right
elif
root.value == '-':
    return
left - right
    elif root.value == '*':
return left * right
elif root.value == '/':
return left / right
#
Inorder Traversal def
inorder(root):
    if root:
if root.left:
        print("(", end="")
inorder(root.left)
print(root.value, end="")
inorder(root.right)
    if
root.right:
        print(")",
end="")
# Main Program postfix = input("Enter
postfix expression: ") root =
build_tree(postfix) print("Inorder
Expression:", end=" ") inorder(root)
print("\nEvaluated Result:", evaluate(root))

```

Result

The **Expression Tree** was successfully constructed from the given **postfix expression** using a stack. The expression tree was traversed to display its infix form, and the expression was evaluated correctly using recursive tree evaluation.

Ex.No: 8 Implement a priority queue using min-heap or max-heap

Aim

To implement a **Priority Queue** using a **Min-Heap** in Python and perform insertion, deletion, and display operations.

Procedure

1. Import the heapq module, which provides an efficient implementation of a min-heap.
2. Create an empty list to represent the priority queue.
3. Implement the following operations:
 - o **Insert (Enqueue):** Add an element to the heap using heappush().
 - o **Delete (Dequeue):** Remove the element with the highest priority (smallest value) using heappop().
 - o **Display:** Show the elements currently present in the heap.
4. Insert multiple elements into the priority queue.
5. Display the queue after insertion.
6. Remove the highest-priority element.
7. Display the updated priority queue.

Program: import

heapq

Priority Queue using Min-Heap

```
class PriorityQueue:    def
```

```
__init__(self):        self.heap = []
```

```
def insert(self, item):
```

```
    heapq.heappush(self.heap, item)
```

```
print(item, "inserted.")    def
```

```
delete(self):           if len(self.heap) ==
```

```
0:                       print("Priority Queue is
```

```
Empty")
```

```
    else:
```

```

        print(heapq.heappop(self.heap), "removed.")
def display(self):    if len(self.heap) == 0:
    print("Priority Queue is Empty")
    else:
        print("Priority Queue:", self.heap)
# Main Program pq = PriorityQueue()
pq.insert(25) pq.insert(10) pq.insert(30)
pq.insert(5) pq.insert(15) print("\nAfter
Insertion:") pq.display() print("\nDeleting
Highest Priority Element:") pq.delete()
print("\nAfter Deletion:") pq.display()

```

Result

A **Priority Queue** was successfully implemented using a **Min-Heap** in Python. The operations **insert**, **delete**, and **display** were performed successfully. The element with the **highest priority (smallest value)** was removed first, demonstrating the correct behavior of a min-heap-based priority queue.

Ex.No: 9 Implement Dijkstra's algorithm for shortest path detection.

Aim

To implement **Dijkstra's Algorithm** in Python to find the **shortest path** from a source vertex to all other vertices in a weighted graph.

Procedure

1. Represent the graph using an adjacency list.
2. Initialize the distance of all vertices as infinity (∞), except the source vertex, whose distance is set to 0.
3. Create a priority queue (min-heap) and insert the source vertex.
4. While the priority queue is not empty:
 - o Remove the vertex with the minimum distance.
 - o For each adjacent vertex:

- Calculate the new distance.
- If the new distance is smaller than the existing distance:
 - Update the distance.
 - Insert the updated vertex into the priority queue.
- 5. Repeat until all reachable vertices are processed.
- 6. Display the shortest distance from the source vertex to every other vertex.

```

Program: import heapq # Dijkstra's Algorithm
def dijkstra(graph, start):
    distances = {vertex: float('inf') for vertex in graph}
    distances[start] = 0
    priority_queue = [(0, start)]
    while priority_queue:
        current_distance, current_vertex = heapq.heappop(priority_queue)
        if current_distance > distances[current_vertex]:
            continue
        for neighbor, weight in graph[current_vertex]:
            distance = current_distance + weight
            if distance < distances[neighbor]:
                distances[neighbor] = distance
        heapq.heappush(priority_queue, (distance, neighbor))
    return distances

# Graph Representation
graph = {
    'A': [('B', 4), ('C', 2)],
    'B': [('A', 4), ('C', 1), ('D', 5)],
    'C': [('A', 2), ('B', 1), ('D', 8), ('E', 10)],
    'D': [('B', 5), ('C', 8), ('E', 2), ('F', 6)],
    'E': [('C', 10), ('D', 2), ('F', 3)],
    'F': [('D', 6), ('E', 3)]
}
source = 'A'
result = dijkstra(graph, source)
print("Shortest distances from source vertex", source)
for vertex in result:
    print(source, "->", vertex, "=", result[vertex])

```


Result

The **Dijkstra's Algorithm** was successfully implemented in Python. The program correctly computed the **shortest distances** from the source vertex to all other vertices in the weighted graph using a **priority queue (min-heap)**.

Ex.No : 10 Implement Prim's and Kruskal's algorithms to find MST.

Aim

To implement **Prim's Algorithm** and **Kruskal's Algorithm** in Python to find the **Minimum Spanning Tree (MST)** of a weighted undirected graph.

Procedure

Prim's Algorithm

1. Represent the graph using an adjacency list.
2. Select any vertex as the starting vertex.
3. Initialize a **min-heap (priority queue)** and mark the starting vertex as visited.
4. Add all adjacent edges of the starting vertex to the priority queue.
5. Repeatedly remove the edge with the minimum weight.
6. If the destination vertex is not visited:
 - o Add the edge to the MST. o Mark the vertex as visited.
 - o Insert all adjacent edges of the newly visited vertex into the priority queue.
7. Continue until all vertices are included in the MST.
8. Display the edges and total cost of the MST.

Kruskal's Algorithm

1. Represent the graph as a list of weighted edges.
2. Sort all edges in ascending order of weight.
3. Initialize each vertex as its own parent using the **Disjoint Set (Union-Find)** data structure.
4. Select the smallest edge.
5. If the edge does not form a cycle:

- o Add it to the MST.
 - o Perform the union operation.
6. Repeat until **(V-1)** edges are included in the MST.
 7. Display the edges and total cost of the MST.

Program: import

```

heapq
# -----
# Prim's Algorithm
# ----- def
prim(graph, start):
    visited = set()    min_heap =
    [(0, start, None)]    mst = []
    total_cost = 0    while
    min_heap:
        weight, current, parent = heapq.heappop(min_heap)
        if current in visited:            continue
    visited.add(current)    if parent is not
    None:            mst.append((parent, current,
    weight))            total_cost += weight
    for neighbor, wt in graph[current]:            if
    neighbor not in visited:
        heapq.heappush(min_heap, (wt, neighbor, current))
    return mst, total_cost
# -----
# Kruskal's Algorithm
# -----
class DisjointSet:    def
    __init__(self, vertices):
        self.parent = {v: v for v in vertices}
    def find(self, item):

```

```

        if self.parent[item] != item:
self.parent[item] = self.find(self.parent[item])
return self.parent[item]    def union(self, x, y):
root1 = self.find(x)    root2 = self.find(y)    if
root1 != root2:        self.parent[root2] = root1
return True    return False def kruskal(vertices,
edges):    ds = DisjointSet(vertices)
edges.sort(key=lambda x: x[2])    mst = []
total_cost = 0    for u, v, w in edges:        if
ds.union(u, v):        mst.append((u, v, w))
total_cost += w    return mst, total_cost

# -----
# Graph
# ----- graph
= {
    'A': [('B', 2), ('C', 3)],
    'B': [('A', 2), ('C', 1), ('D', 4)],
    'C': [('A', 3), ('B', 1), ('D', 5), ('E', 6)],
    'D': [('B', 4), ('C', 5), ('E', 7)],
    'E': [('C', 6), ('D', 7)]
}
vertices = ['A', 'B', 'C', 'D', 'E'] edges
= [
    ('A', 'B', 2),
    ('A', 'C', 3),
    ('B', 'C', 1),
    ('B', 'D', 4),
    ('C', 'D', 5),
    ('C', 'E', 6),
    ('D', 'E', 7)

```

```

]
# Prim's Algorithm mst1, cost1 =
prim(graph, 'A') print("Prim's Minimum
Spanning Tree:") for u, v, w in mst1:
    print(u, "-", v, ":", w)
print("Total Cost =", cost1) print()
# Kruskal's Algorithm mst2, cost2 =
kruskal(vertices, edges) print("Kruskal's
Minimum Spanning Tree:") for u, v, w in
mst2:
    print(u, "-", v, ":", w) print("Total
Cost =", cost2)

```

Result

The **Prim's Algorithm** and **Kruskal's Algorithm** were successfully implemented in Python. Both algorithms correctly generated the **Minimum Spanning Tree (MST)** for the given weighted undirected graph and produced the same **minimum total cost of 13**, confirming the correctness of the implementations.

Ex.No: 11 Implement and compare quick sort, merge sort, and heap sort.

Aim

To implement **Quick Sort**, **Merge Sort**, and **Heap Sort** in Python and compare their outputs by sorting a given list of elements.

Procedure

Quick Sort

1. Select the last element as the pivot.
2. Partition the array such that elements smaller than the pivot are placed before it and larger elements after it.
3. Recursively apply the same process to the left and right subarrays.
4. Display the sorted array.

Merge Sort

1. Divide the array into two halves.
2. Recursively sort both halves.
3. Merge the sorted halves into a single sorted array.
4. Display the sorted array.

Heap Sort

1. Build a max-heap from the input array.
2. Swap the root (largest element) with the last element.
3. Reduce the heap size by one.
4. Heapify the root again.
5. Repeat until the array is completely sorted.
6. Display the sorted array.

Program:

```
# -----  
  
# Quick Sort  
  
# -----  
  
def quick_sort(arr):    if len(arr) <= 1:        return arr  
    pivot = arr[-1]    left = [x for x in arr[:-1] if x <=  
    pivot]    right = [x for x in arr[:-1] if x > pivot]  
    return quick_sort(left) + [pivot] + quick_sort(right)  
  
# -----  
  
# Merge Sort  
  
# ----- def  
  
merge_sort(arr):    if len(arr)  
<= 1:        return arr    mid =
```

```

len(arr) // 2    left =
merge_sort(arr[:mid])    right =
merge_sort(arr[mid:])    return
merge(left, right) def
merge(left, right):
    result = []    i = j = 0    while i <
len(left) and j < len(right):        if
left[i] < right[j]:
result.append(left[i])                i += 1
else:
    result.append(right[j])
j += 1    result.extend(left[i:])
result.extend(right[j:])
return result

# -----
# Heap Sort
# ----- def
heapify(arr, n, i):    largest = i    left =
2 * i + 1    right = 2 * i + 2    if left <
n and arr[left] > arr[largest]:
    largest = left    if right < n and
arr[right] > arr[largest]:
    largest = right
if largest != i:
    arr[i], arr[largest] = arr[largest], arr[i]
heapify(arr, n, largest) def heap_sort(arr):
n = len(arr)    for i in range(n // 2 - 1, -1, -
1):
    heapify(arr, n, i)    for i in
range(n - 1, 0, -1):        arr[0],

```

```

arr[i] = arr[i], arr[0]
heapify(arr, i, 0)    return arr
# -----
# Main Program
# ----- data = [45,
12, 78, 23, 56, 89, 11] print("Original
List:", data) print("\nQuick Sort :",
quick_sort(data)) print("Merge Sort :",
merge_sort(data)) heap_data =
data.copy()

print("Heap Sort :", heap_sort(heap_data))

```

Result

The **Quick Sort**, **Merge Sort**, and **Heap Sort** algorithms were successfully implemented in Python. All three algorithms correctly sorted the given list in ascending order.

Ex.No 12 Implement binary search and compare with linear search.

Aim

To implement **Linear Search** and **Binary Search** in Python and compare their performance in searching for an element in a list.

Procedure

Linear Search

1. Read a list of elements.
2. Read the element to be searched.
3. Traverse the list from the beginning.
4. Compare each element with the search key. 5. If the element is found, display its position.
6. Otherwise, display that the element is not found.

Binary Search

1. Arrange the elements in sorted order.
2. Read the search key.

3. Initialize low and high indices.
4. Find the middle element.
5. Compare the middle element with the search key:
 - o If equal, display the position.
 - o If the key is smaller, search the left half.
 - o If the key is larger, search the right half.
6. Repeat until the element is found or the search interval becomes empty.
7. Display the result.

Program: # Linear Search

```
def linear_search(arr, key):
```

```
    for i in range(len(arr)):
```

```
        if arr[i] == key:
```

```
            return i
```

```
    return -1
```

Binary Search def

```
binary_search(arr, key):
```

```
    low = 0    high =
```

```
    len(arr) - 1    while
```

```
    low <= high:
```

```
        mid = (low + high) // 2
```

```
    if arr[mid] == key:
```

```
        return mid
```

```
    elif arr[mid] < key:
```

```
        low = mid + 1
```

```
    else:        high = mid
```

```
    - 1    return -1
```

Main Program

```
arr = [10, 20, 30, 40, 50, 60, 70, 80] key =
```

```
int(input("Enter element to search: "))
```



```
# Linear Search pos1 =
linear_search(arr, key) if
pos1 != -1:
    print("Linear Search: Element found at index", pos1) else:
    print("Linear Search: Element not found")
# Binary Search pos2 =
binary_search(arr, key) if
pos2 != -1:
    print("Binary Search: Element found at index", pos2) else:
    print("Binary Search: Element not found")
```

Result

The **Linear Search** and **Binary Search** algorithms were successfully implemented in Python. Both algorithms correctly searched for the given element. **Linear Search** can be applied to both sorted and unsorted lists but has a time complexity of **O(n)**, whereas **Binary Search** requires a **sorted list** and performs significantly faster with a time complexity of **O(log n)**, making it more suitable for large datasets.

Ex.No: 13 Implement a hash table with chaining or open addressing.

Aim

To implement a **Hash Table** using the **Chaining** collision resolution technique in Python and perform insertion, search, deletion, and display operations.

Procedure

1. Create a hash table of fixed size using a list of empty lists (buckets).
2. Define a hash function to compute the index:
 - o $\text{index} = \text{key} \% \text{table_size}$
3. Implement the following operations:
 - o **Insert:** Compute the hash index and insert the key into the corresponding bucket.
 - o **Search:** Compute the hash index and search the key in the bucket.
 - o **Delete:** Compute the hash index and remove the key if it exists.
 - o **Display:** Display all buckets in the hash table.
4. Insert sample keys into the hash table.

5. Search for a key.
6. Delete a key.
7. Display the updated hash table.

Program:

```
# Hash Table using Chaining class
HashTable:

def __init__(self, size):
    self.size = size    self.table = [[]
for _ in range(size)]

# Hash Function    def
hash_function(self, key):
    return key % self.size    #
Insert    def insert(self, key):
    index = self.hash_function(key)
    self.table[index].append(key)
    print(key, "inserted.")    # Search
    def search(self, key):
        index = self.hash_function(key)
        if key in self.table[index]:
            return True
        return False    #
Delete    def
delete(self, key):
    index = self.hash_function(key)
    if key in self.table[index]:
        self.table[index].remove(key)
    print(key, "deleted.")    else:
        print(key, "not found.")
# Display    def display(self):
    print("\nHash Table")    for
```

```

i in range(self.size):
print(i, ":", self.table[i])
# Main Program ht =
HashTable(7) keys = [15, 22, 7,
29, 18, 35, 44] for key in keys:
ht.insert(key)
ht.display() key
= 29 if
ht.search(key):
    print("\n", key, "found in hash table.") else:
    print("\n", key, "not found.")
ht.delete(22) ht.display()

```

Result

The **Hash Table** was successfully implemented using the **Chaining** collision resolution technique in Python. The operations **insert**, **search**, **delete**, and **display** were performed successfully, and collisions were effectively handled by storing multiple keys in the same bucket using linked lists (Python lists).

Ex.No 14: Solving string analysis problems using hash structures.

Aim

To solve **string analysis problems** using **hash structures (dictionary/hash map)** in Python by calculating the frequency of characters and identifying duplicate characters in a given string.

Procedure

1. Read a string from the user.
2. Create an empty hash table using a Python dictionary.
3. Traverse each character in the string.
4. If the character already exists in the hash table, increment its count.
5. Otherwise, insert the character into the hash table with a count of 1.
6. Display the frequency of each character.
7. Identify and display the duplicate characters (frequency greater than 1).
8. Display the total number of unique characters.

Program:

```
# String Analysis Using Hash Structures def
string_analysis(text):
    hash_table = {}
    # Count character frequencies
    for ch in text:    if ch in
```

```

hash_table:
hash_table[ch] += 1      else:
    hash_table[ch] = 1
print("\nCharacter Frequencies:")    for
key, value in hash_table.items():
print(f'{key} : {value}')
print("\nDuplicate Characters:")
found = False    for key, value in
hash_table.items():    if value > 1:
    print(f'{key} : {value}')
found = True    if not found:
    print("No duplicate characters found.")
print("\nTotal Unique Characters:", len(hash_table))
# Main Program
text = input("Enter a string: ") string_analysis(text)

```

Result

The program successfully used a **hash structure (dictionary)** to analyze a string. It correctly computed the **frequency of each character**, identified **duplicate characters**, and determined the **total number of unique characters**, demonstrating the efficiency of hash-based string analysis with an average time complexity of **O(n)**.

Ex.No: 15 Implement first-fit and best-fit memory allocation strategies

Aim

To implement **First-Fit** and **Best-Fit** memory allocation strategies in Python for allocating processes to memory blocks.

Procedure

First-Fit Algorithm

1. Initialize the available memory blocks and process sizes.
2. For each process:
 - o Scan the memory blocks from the beginning. o Allocate the process to the **first block** that is large enough.
 - o Reduce the remaining size of the allocated block.
3. If no suitable block is found, the process remains unallocated.
4. Display the allocation results.

Best-Fit Algorithm

1. Initialize the available memory blocks and process sizes.
2. For each process:
 - o Search all memory blocks. o Select the **smallest block** that is large enough to accommodate the process. o Allocate the process to that block.
 - o Reduce the remaining size of the allocated block.
3. If no suitable block is found, the process remains unallocated.
4. Display the allocation results. Program:

```
# First-Fit Memory Allocation def
first_fit(blocks, processes):
    allocation = [-1] * len(processes)
    memory = blocks.copy()
    for i in range(len(processes)):
        for j in range(len(memory)):
            if memory[j] >= processes[i]:
                allocation[i] = j
                memory[j]
```

```

    -= processes[i]          break
return allocation
# Best-Fit Memory Allocation def
best_fit(blocks, processes):
    allocation = [-1] * len(processes)
    memory = blocks.copy()    for i
    in range(len(processes)):
        best = -1            for j in
        range(len(memory)):    if
        memory[j] >= processes[i]:
        if best == -1 or memory[j] <
        memory[best]:          best =
        j            if best != -1:
            allocation[i] = best
    memory[best] -= processes[i]    return
    allocation
# Main Program blocks = [100,
500, 200, 300, 600] processes =
[212, 417, 112, 426]
print("Memory Blocks:", blocks)
print("Processes:", processes)
# First-Fit print("\nFirst-Fit
Allocation") ff =
first_fit(blocks, processes) for
i in range(len(processes)):    if
ff[i] != -1:
    print("Process", i + 1, "allocated to Block", ff[i] + 1)
else:
    print("Process", i + 1, "Not Allocated")
# Best-Fit print("\nBest-Fit
Allocation") bf =
best_fit(blocks, processes) for
i in range(len(processes)):    if
bf[i] != -1:
    print("Process", i + 1, "allocated to Block", bf[i] + 1)
else:
    print("Process", i + 1, "Not Allocated")

```

Result

The **First-Fit** and **Best-Fit** memory allocation strategies were successfully implemented in Python. Both algorithms allocated processes to available memory blocks according to their respective strategies.